SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing Techniques (BL4, BL5)
Assessment Tool Weightage: 25%

QUESTIONS: 05 **Total Marks:25**

Annexure A

Error Analysis Task

Code of Conduct:

I D Varshini / 192320007 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Error Analysis Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Annexure A

Error Analysis Task

Q1. Scenario: Online Banking System

Modules: User Login, Fund Transfer, Account Verification

Question:

Write pseudocode for a quality assurance testing process that validates secure user authentication and fund transfer operations. Include input validation, transaction verification, exception handling, and test result logging.

Answer:

The quality assurance process should test both successful and unsuccessful conditions. Authentication tests must reject incomplete or invalid credentials, while fund-transfer tests must verify account ownership, beneficiary details, amount limits, balance sufficiency, and the final transaction state. Every test should have an expected result so that the actual system response can be compared objectively.

Pseudocode:

```
BEGIN QA_BANKING_TEST
  INITIALIZE testResults, logger
  DEFINE loginTests = {valid login, wrong password, empty fields, locked user}
  DEFINE transferTests = {valid transfer, invalid account, zero/negative amount,
insufficient balance, invalid beneficiary}

  FOR each test IN loginTests DO
    TRY
      IF username is empty OR password is empty THEN
        actualResult <- "Rejected: Missing input"
      ELSE
        actualResult <- Authenticate(username, password)
      END IF
      COMPARE actualResult WITH expectedResult
      LOG test name, expected result, actual result, PASS/FAIL
    CATCH AuthenticationException e
      LOG test name, error type, FAIL
```

```
    END TRY
  END FOR

  loginStatus <- Authenticate(validUser, validPassword)
  IF loginStatus = SUCCESS THEN
    FOR each test IN transferTests DO
      TRY
        IF sourceAccount invalid OR beneficiary invalid THEN
          actualResult <- "Rejected: Invalid account data"
        ELSE IF amount <= 0 THEN
          actualResult <- "Rejected: Invalid amount"
        ELSE IF amount > availableBalance THEN
          actualResult <- "Rejected: Insufficient balance"
        ELSE
          oldSourceBalance <- GetBalance(sourceAccount)
          oldTargetBalance <- GetBalance(beneficiary)
          transactionId <- Transfer(sourceAccount, beneficiary, amount)
          newSourceBalance <- GetBalance(sourceAccount)
          newTargetBalance <- GetBalance(beneficiary)
          VERIFY transactionId exists
          VERIFY newSourceBalance = oldSourceBalance - amount
          VERIFY newTargetBalance = oldTargetBalance + amount
          actualResult <- "Transfer verified"
        END IF
        COMPARE actualResult WITH expectedResult
        LOG test name, transactionId if available, PASS/FAIL
      CATCH TransactionException e
        ROLLBACK transaction if required
        LOG test name, safe error message, FAIL
      END TRY
    END FOR
  END IF
  GENERATE summary of passed and failed tests
END QA_BANKING_TEST
```

Explanation:

Input validation prevents malformed requests from reaching authentication or transaction logic. Transaction verification checks that the debit and credit are consistent and that a unique transaction identifier is produced. Exception handling prevents the test run from stopping when one case fails and also verifies that the application fails safely. Test-result logging should record the test case, expected outcome, actual outcome, and status, but it should not store passwords, OTPs, or other sensitive authentication values.

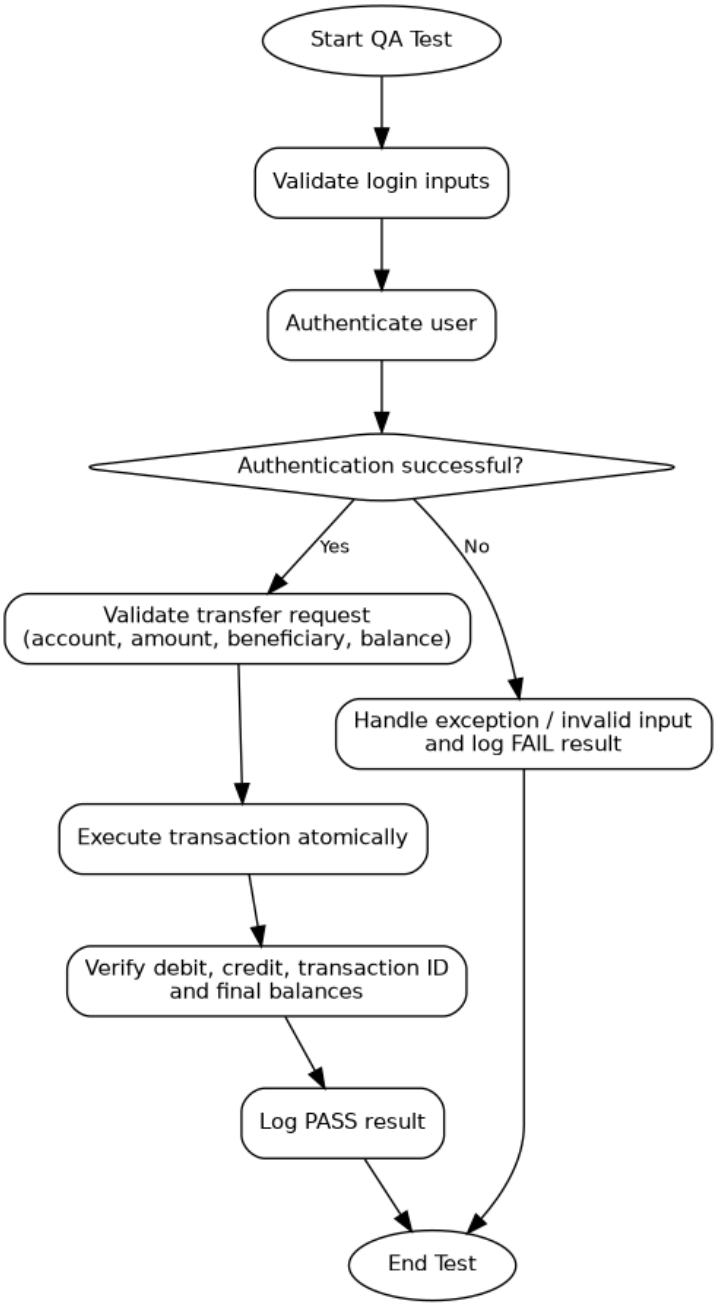

Figure 1. Activity flow for banking QA testing

Q2. Scenario: E-Commerce Shopping Platform

Features: Product Search, Shopping Cart, Payment Gateway

Question:

Design pseudocode for a usability testing evaluation that measures customer interaction efficiency, including product search time, checkout completion time, and navigation complexity. Explain how the algorithm collects and analyzes user behavior data.

Answer:

Usability testing should measure how easily a user completes realistic shopping tasks rather than only checking whether the features technically work. Each participant can be assigned tasks such as finding a product, adding it to the cart, and completing checkout. During the session, the system records task duration, number of clicks or page transitions, backtracking, errors, and successful completion.

Pseudocode:

```
BEGIN ECOMMERCE_USABILITY_TEST
  INITIALIZE sessionData, participantCount
  FOR each participant DO
    CREATE anonymous sessionId
    RESET clicks, pageTransitions, backtracks, errors

    START searchTimer
    ASK participant to locate a specified product
    RECORD search actions, clicks, filters used, backtracks
    STOP searchTimer when correct product is opened
    searchTime <- elapsed time

    ASK participant to add product to cart and begin checkout
    START checkoutTimer
    RECORD checkout clicks, page changes, validation errors, payment errors
    STOP checkoutTimer when order confirmation is displayed or user abandons task
    checkoutTime <- elapsed time

    navigationComplexity <- pageTransitions + backtracks + errors
```

```
        taskSuccess <- TRUE if product found AND checkout completed ELSE FALSE
        STORE sessionId, searchTime, checkoutTime, navigationComplexity, taskSuccess
    END FOR

    avgSearchTime <- AVERAGE(all searchTime values)
    avgCheckoutTime <- AVERAGE(all checkoutTime values)
    avgNavigationComplexity <- AVERAGE(all navigationComplexity values)
    completionRate <- successfulSessions / participantCount * 100
    IDENTIFY screens with highest backtracks, errors, and delays
    GENERATE usability report and improvement recommendations
END ECOMMERCE_USABILITY_TEST
```

Data Collection and Analysis:

The algorithm collects event data automatically from each test session. Timestamps give search and checkout duration, while click and navigation events show how much effort was required. Backtracking and repeated validation errors indicate confusing interfaces. After all sessions, the algorithm calculates averages and completion rate and identifies steps that repeatedly slow users down. For example, a high search time may indicate poor filtering, while a high checkout time with many errors may indicate an unclear payment form. These findings convert observed user behavior into measurable usability improvements.

Q3. Scenario: Hospital Management System

Modules: Patient Registration, Appointment Scheduling, Medical Records

Question:

Write pseudocode to generate and execute test cases for appointment scheduling functionality. Include validation of doctor availability, appointment conflicts, patient information accuracy, and system responses to invalid requests.

Answer:

Appointment scheduling requires both positive and negative test cases. The test generator should cover a valid booking, unavailable doctor, duplicate or overlapping appointment, missing or invalid patient details, past date/time, and invalid doctor or patient identifiers. Each case is executed independently and compared with its expected response.

Pseudocode:

```
BEGIN APPOINTMENT_TEST_SUITE
  testCases <- EMPTY LIST
  ADD case(valid patient, available doctor, free slot, EXPECT success)
  ADD case(valid patient, unavailable doctor, requested slot, EXPECT rejection)
  ADD case(valid patient, doctor already booked, conflicting slot, EXPECT conflict error)
  ADD case(invalid/missing patient data, available doctor, free slot, EXPECT validation
error)
  ADD case(valid patient, invalid doctorId, free slot, EXPECT doctor-not-found error)
  ADD case(valid patient, available doctor, past date/time, EXPECT invalid-date error)

  FOR each tc IN testCases DO
    TRY
      IF NOT ValidatePatient(tc.patient) THEN
        actual <- "Invalid patient information"
      ELSE IF NOT DoctorExists(tc.doctorId) THEN
        actual <- "Doctor not found"
      ELSE IF NOT IsFutureDateTime(tc.dateTime) THEN
        actual <- "Invalid appointment time"
      ELSE IF NOT IsDoctorAvailable(tc.doctorId, tc.dateTime) THEN
        actual <- "Doctor unavailable"
      ELSE IF HasAppointmentConflict(tc.patientId, tc.doctorId, tc.dateTime) THEN
        actual <- "Appointment conflict"
      ELSE
        appointmentId <- CreateAppointment(tc.patientId, tc.doctorId, tc.dateTime)
        actual <- "Success"
        VERIFY appointmentId is created
        VERIFY appointment appears in patient and doctor schedules
      END IF
      status <- PASS if actual matches tc.expected ELSE FAIL
      LOG tc.id, expected, actual, status
    CATCH SystemException e
      LOG tc.id, "Unexpected system exception", FAIL
```

```
        END TRY
    END FOR
    GENERATE test summary
END APPOINTMENT_TEST_SUITE
```

Justification:

Doctor availability and conflict checks prevent double booking. Patient validation ensures that an appointment is linked only to a valid and complete patient record. Negative test cases confirm that invalid requests are rejected with meaningful responses instead of creating partial or inconsistent records. Finally, verifying both the patient schedule and the doctor schedule confirms that the successful transaction updates the system consistently.

Q4. Scenario: Airline Reservation System

Modules: Passenger Registration, Flight Booking, Seat Allocation

Question:

Develop pseudocode for a constraint-based testing approach where booking requests must satisfy conditions such as valid passenger information, seat availability, and payment confirmation. Show how invalid booking scenarios are detected and handled.

Answer:

Constraint-based testing treats each booking rule as a condition that must be satisfied before a reservation is confirmed. A valid request must contain acceptable passenger information, an existing flight, an available seat, and successful payment. The test suite should deliberately violate one constraint at a time so the system response can be checked precisely.

Pseudocode:

```
BEGIN CONSTRAINT_BASED_BOOKING_TEST
    DEFINE constraints:
        C1 = passenger information is complete and valid
        C2 = flight exists and is open for booking
        C3 = requested seat is available
        C4 = payment is confirmed

    testCases <- {
        all constraints valid,
        invalid passenger with C2-C4 valid,
```

```
    unavailable seat with C1,C2,C4 valid,
    failed payment with C1-C3 valid,
    invalid flight with remaining data valid
  }

  FOR each request IN testCases DO
    TRY
      IF NOT ValidatePassenger(request.passenger) THEN
        REJECT booking with "Invalid passenger information"
      ELSE IF NOT FlightIsBookable(request.flightId) THEN
        REJECT booking with "Flight unavailable"
      ELSE IF NOT SeatIsAvailable(request.flightId, request.seatNo) THEN
        REJECT booking with "Seat unavailable"
      ELSE
        TEMPORARILY HOLD requested seat
        paymentStatus <- ProcessPayment(request.paymentData)
        IF paymentStatus != CONFIRMED THEN
          RELEASE held seat
          REJECT booking with "Payment not confirmed"
        ELSE
          bookingId <- ConfirmBooking(request)
          MARK seat as booked
          VERIFY bookingId, passenger mapping, and seat allocation
          RETURN "Booking successful"
        END IF
      END IF
      LOG violated constraint or successful result
    CATCH BookingException e
      RELEASE held seat if necessary
      LOG safe error details and FAIL
    END TRY
  END FOR
END CONSTRAINT_BASED_BOOKING_TEST
```

Invalid Scenario Handling:

Each rejected request is stopped at the first violated constraint and receives a clear error message. No booking record should be finalized for an invalid passenger, unavailable seat, or failed payment. If a seat was temporarily held before payment, it must be released when payment fails or an exception occurs. This prevents seat leakage and inconsistent reservations. Constraint-based tests are useful because they directly connect business rules to expected system behavior.

Q5. Scenario: Smart Library Management System

Features: Book Search, Book Reservation, Borrow/Return Services

Question:

Write pseudocode for integrating quality assurance and usability testing where the system evaluates both functional correctness (reservation and borrowing operations) and user experience factors such as response time, ease of navigation, and accessibility.

Answer:

An integrated test should evaluate whether library operations produce correct results and whether users can perform those operations efficiently. Functional QA verifies search, reservation, borrowing, and return rules. Usability testing measures response time, task completion, navigation effort, and accessibility. Combining both views prevents a system from being technically correct but difficult to use.

Pseudocode:

```
BEGIN INTEGRATED_LIBRARY_TEST
  INITIALIZE functionalResults, usabilityResults

  // Functional QA
  FOR each functionalCase IN {search, reserve, borrow, return} DO
    START responseTimer
    actual <- Execute(functionalCase)
    STOP responseTimer
    VERIFY actual matches expected output
    VERIFY book availability/status is updated correctly
    VERIFY invalid operations are rejected
    STORE functional PASS/FAIL and responseTime
```

```
    END FOR

    // Usability evaluation
    FOR each userTask IN {find book, reserve book, borrow book, return book} DO
      RESET clicks, pageTransitions, errors
      START taskTimer
      OBSERVE user actions until task completes or is abandoned
      STOP taskTimer
      taskSuccess <- TRUE if user completes task correctly ELSE FALSE
      navigationScore <- clicks + pageTransitions + errors
      STORE taskTime, taskSuccess, navigationScore
    END FOR

    // Accessibility checks
    CHECK keyboard navigation for major controls
    CHECK form fields and buttons have meaningful labels
    CHECK error messages are understandable and not dependent only on color
    CHECK focus order and readable text presentation

    avgResponseTime <- AVERAGE(functional response times)
    taskCompletionRate <- successful usability tasks / total tasks * 100
    avgNavigationScore <- AVERAGE(navigation scores)

    IF functional failures exist THEN mark release as NOT READY
    IF response time or usability metrics exceed accepted thresholds THEN flag UX
  improvement
    GENERATE combined QA + usability report
  END INTEGRATED_LIBRARY_TEST
```

Evaluation:

Functional correctness is determined by comparing expected and actual results and by checking the final state of books and reservations. Response time indicates system performance from the user perspective. Ease of navigation can be estimated using clicks, page transitions, errors, and completion rate. Accessibility checks confirm that core tasks are usable with keyboard

navigation and clear labels and feedback. The final report should separate functional defects from usability issues while also identifying cases where both interact, such as a slow reservation screen causing users to repeat an action.

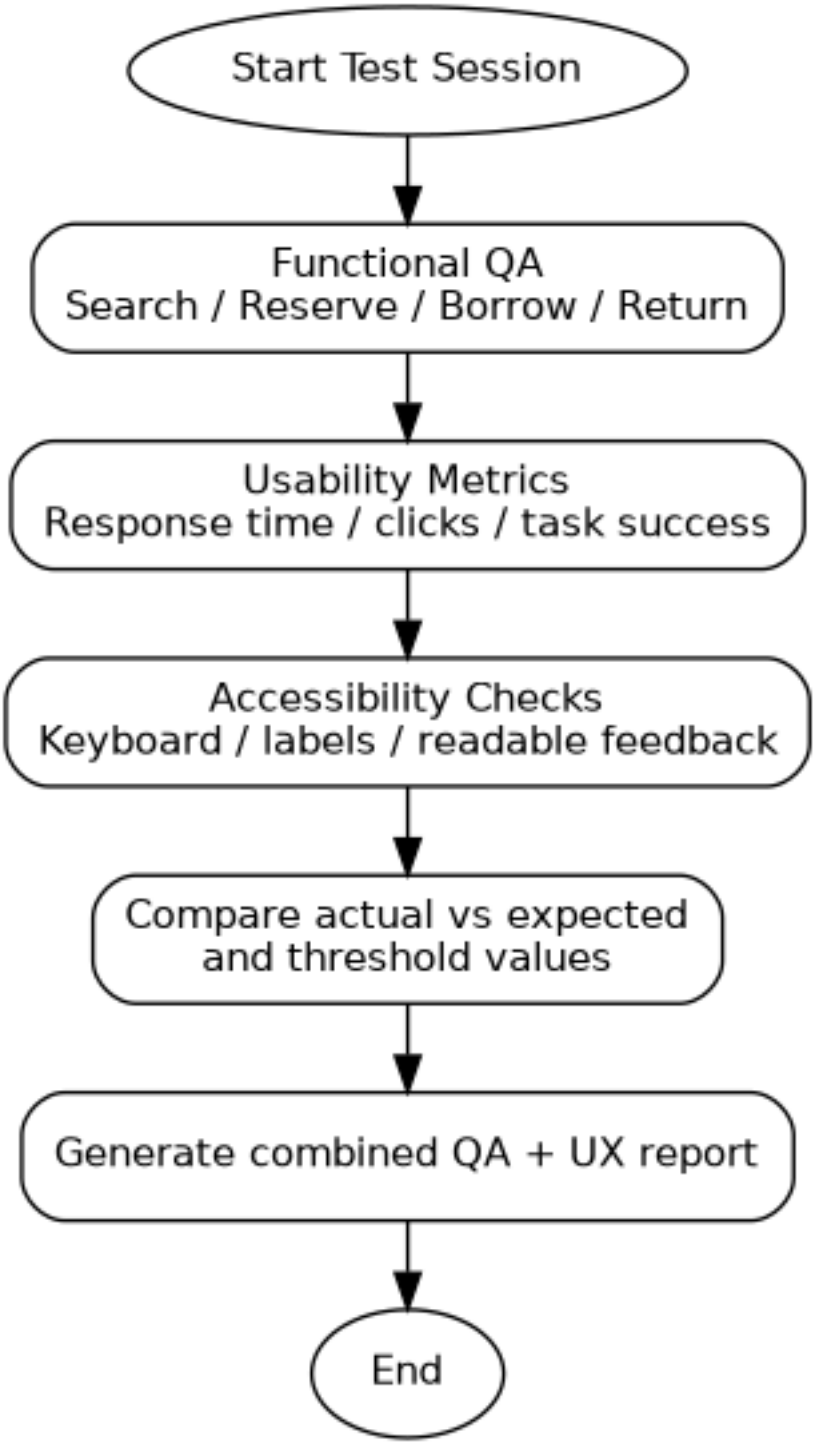

Figure 2. Integrated functional QA and usability-testing flow